Normative Continuity under Self-Revision

How rules can change without erasing authority or obligations

Revision 2 (mathematical settlement) — 30 August 2026

The question

Suppose a reasoner can revise the normative rules it represents: its judgments, warrants, evaluators, adjudicative procedures, and even represented rules about how those things may be revised. No substantive rule has to remain in force forever. What has to stay continuous so that revision does not become a way to create authority from nowhere, erase an inconvenient obligation, change the rules of a pending case, reset its history, or leave it waiting forever?

The proposal in this note is simple in spirit:

the content may change; the history of what was authorized and what became owed may not be silently rewritten.

There are two parts. *Legitimate Evolution* tracks how rules acquire and lose authority. The *Answerable Process* tracks what happens to the issues those rules create. Together they support the headline result, Theorem 4: provided no fixed no-route wait can remain unresolved forever, every matter either closes through recorded resolutions or receives unbounded total attention.

What is and is not fixed. The represented rules may change. The mathematical vocabulary used to read a history does not: for example, the relation $\mathsf{Permit}_n(c)$ is a fixed part of the model which asks what the reasoner's *current represented rules* say about proposed change c. In this sense the reasoner may change its represented rules of revision without changing the metalogical notion of what it means for those current rules to permit a move. The accepted initial state is also fixed as a historical starting point. The theory does not claim to eliminate every fixed meta-level notion.

Kinds of statements. A *semantic judgment* says what the reasoner's current represented rules say at a given history. A *structural requirement* says how the recorded history must respect those judgments through time. Additional assumptions are stated only when a theorem needs something neither supplied by the structural rules nor determined by the represented history.

1 History and the objects that live in it

Definition (history and grounds)**.** At position $n \in \mathbb{N}$ the reasoner appends a finite batch of records

$$e_n = (r_{n,1}, \ldots, r_{n,k_n}), \qquad H_n = (e_0, \ldots, e_{n-1}).$$

H_n is the strict prefix available before position n. Let

$$\mathsf{Past}_n := \{r_{j,\ell} : j < n\}.$$

Grounds for a move at position n are simply a finite set of prior records,

$$g \subseteq_{\text{fin}} \mathsf{Past}_n.$$

Thus a record written in e_n cannot be cited as a prior ground by another record in the same batch.

A record may change which normative rules stand in force, open or resolve an issue, add or withdraw a prerequisite, or mark an issue as a separately protected matter. Record identity is historical: changing how a later state interprets the same content does not create a new occurrence.

1.1 Rules that currently have authority

At each point in the history, the reasoner treats some represented rules as currently authoritative. Let $\mathcal{N}$ be the set of records that can serve as such rules, and let

$$L_n \subseteq \mathcal{N}$$

be the rules in force before position n. We call a rule in L_n *standing*. The rules in force at the beginning are

$$G := L_0.$$

G is only the starting point. A rule in G may later be removed, and a rule not in G may later acquire standing.

Two facts about a standing rule will matter below:

- $\mathsf{Auth}(\lambda)$ means that λ may be cited as authority for changing which rules have standing.
- $\lambda \triangleright (\kappa, \tau, x)$ means that λ authorizes protocol κ to handle a new issue of kind τ about subject x.

These are features of what the represented rule says. They are not additional semantic judgments that vary separately with time.

Definition (current authority to govern an issue)**.** A protocol is authorized to govern a new issue when some rule already in force says that it may. We write

$$\kappa \Vdash_n (\tau, x) \quad \Longleftrightarrow \quad \exists \lambda \in L_n : \ \lambda \triangleright (\kappa, \tau, x).$$

Thus $\kappa \Vdash_n (\tau, x)$ is derived from the rules standing at H_n; it is not a separate source of authority.

1.2 Issues

Let $\mathcal{Q}$ be the set of issue occurrences. An issue q is the occurrence of an issue-opening record. It has immutable opening data

$$\pi(q) = (\tau_q, x_q, \kappa_q, s_q^0) \in \mathcal{T} \times \mathcal{X} \times \mathcal{K} \times \mathcal{S}_{\kappa_q}.$$

The four coordinates answer four different questions:

- τ_q — *what kind of question is this?*
- x_q — *what is the question about?*
- κ_q — *which protocol will judge this particular episode?*
- s_q^0 — *what local memory does that protocol start with?*

The subject x_q may itself be a rule, reason, issue, protocol, event, or process component. If q is still open at H_n, write $s_n(q) \in \mathcal{S}_{\kappa_q}$ for its current local state.

Let $\mathcal{Q}_n^+ \subseteq \mathcal{Q}$ be the issues opened in batch e_n; the superscript $+$ is meant to suggest "new at n." The sets $\mathcal{Q}_n^+$ are disjoint. Thus

$$I_n := \bigcup_{j<n} \mathcal{Q}_j^+$$

is the set of issues incurred before position n, and $O_n \subseteq I_n$ is the set still outstanding at H_n.

1.3 Prerequisites

Let $\mathcal{D}$ be the set of prerequisite occurrences. A prerequisite occurrence is identified with the record that first introduces it, so the same occurrence cannot later be "re-added" after withdrawal. Its immutable data are

$$\delta(d) = (q_d, \chi_d, T_d).$$

Here q_d is the issue that owns the prerequisite, χ_d is the condition that would satisfy it, and $T_d \subseteq_{\text{fin}} \mathcal{Q}$ is a finite set of issue occurrences named as initial internal ways of working toward χ_d. Different prerequisites of one issue are conjunctive; the alternatives in T_d are disjunctive.

For $q \in O_n$, let $\mathcal{P}_n(q) \subseteq \mathcal{D}$ be the finite set of prerequisites currently active for q.

2 What the reasoner's current rules decide

The following are the *only* primitive prefix-dependent semantic judgments in this note. Each is evaluated from H_n and may read the current standing state L_n and whatever represented rules are present in H_n. Everything introduced later is derived from the history plus these judgments.

Semantic judgment (permission to change standing)**.** $\mathsf{Permit}_n(c)$ means that the current represented rules permit proposed standing-change record c at H_n.

Semantic judgment (what is currently due)**.** The current rules determine a set

$$\mathsf{Due}_n := \mathsf{Due}(H_n, L_n) \subseteq \mathcal{T} \times \mathcal{X}.$$

What creates a new obligation is a rising edge,

$$\mathsf{NewDue}_n := \begin{cases} \mathsf{Due}_0, & n = 0, \\ \mathsf{Due}_n \setminus \mathsf{Due}_{n-1}, & n > 0. \end{cases}$$

If a pair later drops out of Due_n, that does not close any issue already opened for it. If it later reappears, that is a new rising edge and can create a new issue.

Semantic judgment (resolution)**.** For $q \in O_n$, $g \subseteq_{\text{fin}} \mathsf{Past}_n$, and finite $S \subseteq \mathcal{Q}$, $\mathsf{Resolve}_n(q; g, S)$ means that q's own anchored protocol κ_q, reading H_n and $s_n(q)$, accepts disposing of q into successor set S on grounds g. $S = \varnothing$ is a terminal resolution. $\mathsf{Resolve}_n$ is sufficient on its own: a resolution is not additionally gated by Permit_n. A represented rule may make κ_q's acceptance consult the standing state or a permission judgment, but that is content of the semantics, not a second structural gate; no theorem below reads such a gate, so the interface is kept at the weaker primitive (superseding the frozen kernel's entry A21). If batch e_n actually records such an accepted resolution, write

$$q \overset{g}{\Longrightarrow}_n S, \qquad \mathsf{Res}_n := \{q \in O_n : \exists g, S \; q \overset{g}{\Longrightarrow}_n S\}.$$

Semantic judgment (state continuation)**.** For finite parent set $P \subseteq O_n$, proposed fresh successor q', and grounds g, $\mathsf{Continue}_n(P; q', g)$ means that the current rules accept the proposed initial state $s^0_{q'}$ as a continuation of the parents' current states. This relation can allow ordinary state transport or an explicit reset; what matters structurally is that the transition is represented and accepted, not silent.

Semantic judgment (prerequisite satisfaction)**.** $\mathsf{Met}_n(d)$ means that the condition χ_d is satisfied at prefix H_n. This judgment applies to every prerequisite, not only to information expected from outside the reasoner.

Semantic judgment (adding and withdrawing prerequisites)**.** $\mathsf{AddPre}_n(q; g, d)$ means that the current rules accept introducing fresh prerequisite occurrence d for issue q (so $q_d = q$); $\mathsf{DropPre}_n(q; g, d)$ means that they accept withdrawing active prerequisite d from q. Let $\mathcal{P}^+_n(q)$ and $\mathcal{P}^-_n(q)$ be the prerequisite occurrences actually added and withdrawn by accepted records in e_n; every $d \in \mathcal{P}^+_n(q)$ has $q_d = q$.

Semantic judgment (creating an additional matter)**.** $\mathsf{Designate}_n(q; g)$ means that the current rules accept making issue q the root of an additional protected matter. The issue may already exist or may be opened in the same batch.

For emphasis, the complete list is

$$\mathsf{Permit},\ \mathsf{Due},\ \mathsf{Resolve},\ \mathsf{Continue},\ \mathsf{Met},\ \mathsf{AddPre},\ \mathsf{DropPre},\ \mathsf{Designate}.$$

Relations such as $\Vdash_n$, readiness, successor ancestry, routes, reachability, available work, and service are defined from the record and are not further semantic inputs.

Acceptance convention. For the semantic judgments that govern recorded moves, an *accepted* move means both that its record occurs in e_n and that the corresponding judgment holds at the strict prefix H_n: Permit for standing changes, Resolve for resolutions, AddPre/DropPre for prerequisite changes, and Designate for matter designation. Continue is checked when a successor opens; Due and Met describe the current semantic state rather than recorded moves.

All eight judgments read the strict prefix uniformly. This supersedes the frozen Legitimate Evolution convention (its entry A34), under which Due read descriptive material from the current event while normative standing was read from the pre-state: a record written in e_n here first affects Due at $n+1$. Nothing is lost but one position, and the descriptive/normative split disappears from the interface.

3 How authority changes: Legitimate Evolution

The rules in force are not fixed. At position n the reasoner may add rules to L_n or remove rules from it. Legitimate Evolution imposes one continuity condition on this freedom: a change in authority must answer to authority that was already in force before the change.

Concretely, a proposed change must pass the current rules' own test for whether it is permitted. And if the change actually adds or removes authority, it must cite at least one rule that was already standing and was itself allowed to justify changes of authority.

Structural requirement 1 (changing which rules have authority)**.** Let L^+_n be the rules that gain standing in batch e_n, and let L^-_n be the rules that lose it. Every record c in e_n that proposes such a change must satisfy

$$\mathsf{Permit}_n(c).$$

If c actually adds or removes standing, then

$$\varnothing \neq g(c) \subseteq \{\lambda \in L_n : \mathsf{Auth}(\lambda)\}.$$

After the batch, the rules in force are exactly

$$L_{n+1} = (L_n \setminus L_n^-) \cup L_n^+.$$

As in the frozen kernel, the occurrences in L_n^+ are fresh: each is first recorded in e_n. A rule that has lost standing does not re-enter as the same occurrence; readopting its content produces a new occurrence with its own grounds. A record that neither adds nor removes standing is a no-op and its grounds are unconstrained; the frozen kernel's S1 constrained the grounds of every accepted edit, and the difference has no structural content, because only a change creates a node of an authorization tree.

Because grounds must come from Past_n, every cited authorizer predates the change it justifies. In particular, a rule first added in e_n cannot justify its own admission or another change in the same batch. There is no same-batch bootstrap of authority.

This local requirement has a global consequence. Pick any rule that is in force now and ask what earlier standing rules authorized its admission. Ask the same question about those authorizers, and continue backward. Each step moves to an earlier position, so this history cannot continue forever.

Let the *admitted* occurrences be those that ever gained standing,

$$\mathsf{Adm}_n := G \cup \bigcup_{j<n} L_j^+,$$

so $L_n \subseteq \mathsf{Adm}_n$ and a rule removed from standing stays admitted.

Theorem 1 (Grounded Replay)**.** *Assume Requirement 1. For every $\lambda \in \mathsf{Adm}_n$, there is a finite authorization tree rooted at λ whose leaves lie in G. Each non-leaf node is linked to the standing authorizers cited when that rule entered L, and positions strictly decrease along every branch. In particular this holds for every $\lambda \in L_n$.*

Proof. Induct on n. $\mathsf{Adm}_0 = G$. A rule in $\mathsf{Adm}_{n+1} \setminus \mathsf{Adm}_n$ entered standing in e_n through a change whose grounds are standing authorizers at H_n, hence in Adm_n with trees by the induction hypothesis; hang those trees under the new node at position n. Freshness of L_n^+ makes "the change through which the rule entered" unique; the existence of the tree does not use it. □

The theorem is stated over admitted occurrences because it is a historical statement: it is exactly "ancestry, not permanence." The standing form is the corollary its consumer (Requirement 2) needs.

Grounded Replay is about where current authority came from, not about keeping old rules forever. A rule in the ancestry—even a rule in G—may later be legitimately removed. What survives revision is the historical chain by which present authority was acquired.

This grounding requirement is specific to changes of authority. The structural theory does not require every resolution, prerequisite change, or matter designation to cite nonempty grounds. Those moves may have empty grounds unless their own semantic judgment rules that out. Grounded Replay is therefore a theorem about the history of authority, not a universal grounding theorem for every recorded move.

4 Continuity of issues

The next requirements say what happens once the current rules make something due and an issue has been opened for it.

Structural requirement 2 (standing at opening)**.** Every fresh issue uses a protocol that currently has standing for its kind and subject:

$$q \in \mathcal{Q}_n^+ \quad \Longrightarrow \quad \kappa_q \Vdash_n (\tau_q, x_q).$$

A standing change recorded in the same batch cannot authorize the opening: standing is read from the strict prefix H_n.

Structural requirement 3 (new demands become issues)**.** Every newly due pair is represented by a fresh issue:

$$(\tau, x) \in \mathsf{NewDue}_n \quad \Longrightarrow \quad \exists q \in \mathcal{Q}_n^+ : \ (\tau_q, x_q) = (\tau, x).$$

Additional assumption (compatibility of what is due with what has standing)**.** Whenever $(\tau, x) \in \mathsf{NewDue}_n$, at least one protocol currently has standing for that pair:

$$(\tau, x) \in \mathsf{NewDue}_n \quad \Longrightarrow \quad \exists \kappa \in \mathcal{K} : \ \kappa \Vdash_n (\tau, x).$$

This is not a theorem of the structural bookkeeping. It is the minimal compatibility needed for the previous two requirements to be jointly satisfiable.

Structural requirement 4 (resolution continuity)**.**

$$O_{n+1} = (O_n \setminus \mathsf{Res}_n) \cup \mathcal{Q}_n^+.$$

So an old issue remains outstanding unless it is explicitly resolved, and every fresh issue is outstanding at the next prefix. In particular,

$$q \in O_n \setminus O_{n+1} \quad \Longrightarrow \quad q \in \mathsf{Res}_n.$$

A change in the surrounding normative rules cannot by itself make an issue disappear. Because $\mathsf{Res}_n \subseteq O_n$ and $\mathcal{Q}_n^+ \cap O_n = \varnothing$, an issue cannot be opened and resolved in the same batch: every fresh issue is outstanding for at least one position. This deliberately supersedes the frozen kernel's entry A17. The reason is timing discipline, not content: every judgment about q reads a prefix at which q is outstanding, so $s_n(q)$, $\mathcal{P}_n(q)$ and $\mathsf{Ready}_n(q)$ are defined whenever $\mathsf{Resolve}_n(q; \cdot)$ is consulted; allowing a birth-batch resolution would need a separate birth-state semantics, and would let Requirement 3 be met by an issue that is never outstanding at any prefix.

Structural requirement 5 (successors are fresh)**.** If $q \overset{g}{\Longrightarrow}_n S$, then $S \subseteq \mathcal{Q}_n^+$. Define the successor relation

$$q \to q' \quad \Longleftrightarrow \quad \exists n, g, S \left(q \overset{g}{\Longrightarrow}_n S \ \wedge \ q' \in S \right).$$

Because successors open later than their parents, $\to$ is acyclic. If $q' \in \mathcal{Q}_n^+$, let

$$\mathsf{Par}_n(q') := \{ q \in O_n : \exists g, S \ (q \overset{g}{\Longrightarrow}_n S \wedge q' \in S) \}$$

be the set of parents that resolve into it in that batch.

Freshness of successors deliberately supersedes the frozen kernel's entries A5 and A11, which allowed a resolution to carry into an issue already outstanding. The mathematical reason is prefix-determinacy. With a static successor relation into a preexisting q', $m \preceq q'$ would hold at every

prefix, so $\mathsf{Live}_k(m)$ and $o_k(m)$ for $k < n$ would change when e_n is written: opportunity would not be a function of the history up to k. Time-stamped successor edges repair this at the cost of time-relative ancestry, and buy nothing this theory needs: relating an outstanding issue q to an existing q' is expressed by a prerequisite of q routed to q', which keeps m's identity, puts q' in $\mathsf{Reach}_n(m)$, and so credits q''s work to m (Section 6). Succession is reserved for diachronic continuation of an entitlement into a fresh episode.

Lemma 1 (revision is prospective)**.** *If $q \to q'$, then the resolution of q is judged under κ_q, while q' is governed by its own opening anchor $\kappa_{q'}$, which must stand when q' opens.*

Proof. The resolution record for q can be accepted only through $\mathsf{Resolve}_n(q; g, S)$, whose definition uses the immutable anchor κ_q. By Requirement 5, q' is a fresh issue in the same batch, and Requirement 2 checks its own anchor $\kappa_{q'}$ against the standing state already present in H_n. Thus revision can govern the fresh successor without changing the adjudicative terms of the old issue. □

Thus a protocol can gain or lose standing without changing the rules of a case that is already open. To move a continuing issue to new terms, the process must resolve the old issue and open a fresh successor under those terms.

Structural requirement 6 (state continuity)**.** Every fresh successor with at least one parent begins from an explicitly accepted continuation of the parents' current states:

$$q' \in \mathcal{Q}_n^+,\ \mathsf{Par}_n(q') \neq \varnothing \quad \Longrightarrow \quad \exists g \subseteq_{\text{fin}} \mathsf{Past}_n :\ \mathsf{Continue}_n(\mathsf{Par}_n(q'); q', g).$$

The current rules may allow a reset, but the reset must be represented as the accepted transition to the successor; accumulated local state cannot simply be replaced without such a transition.

5 Prerequisites and matters

5.1 Prerequisites persist until something happens to them

Structural requirement 7 (prerequisite continuity)**.** If q remains outstanding across position n, then

$$\mathcal{P}_{n+1}(q) = \big(\mathcal{P}_n(q) \setminus \mathcal{P}_n^-(q)\big) \cup \mathcal{P}_n^+(q).$$

If $q \in \mathcal{Q}_n^+$ is fresh, then

$$\mathcal{P}_{n+1}(q) = \mathcal{P}_n^+(q).$$

Thus an active prerequisite of a continuing issue remains active unless an accepted withdrawal removes it. Resolving the issue ends its remaining prerequisites with that episode. Because prerequisite identities are fresh record occurrences, a withdrawn d cannot later be reactivated; a later prerequisite is a new occurrence d'.

Structural requirement 8 (prerequisites cannot name future issues)**.** If d is first introduced in e_n, then

$$q_d \in O_n \cup \mathcal{Q}_n^+, \qquad T_d \subseteq O_n \cup \mathcal{Q}_n^+.$$

A prerequisite may point to issues already open or co-opened in the same batch, but not to an issue occurrence that does not yet exist.

Structural requirement 9 (satisfaction is persistent)**.**

$$\mathsf{Met}_n(d) \quad \Longrightarrow \quad \mathsf{Met}_{n+1}(d).$$

A fixed prerequisite may become satisfied; it cannot later become unsatisfied.

Definition (ready issue)**.**

$$\mathsf{Ready}_n(q) \quad \Longleftrightarrow \quad \forall d \in \mathcal{P}_n(q),\ \mathsf{Met}_n(d).$$

Structural requirement 10 (only ready issues may resolve)**.**

$$q \in \mathsf{Res}_n \quad \Longrightarrow \quad \mathsf{Ready}_n(q).$$

Met is a primitive judgment on the condition χ_d. In particular a route's terminal resolution does not by itself satisfy the prerequisite it was a route for; whether it does is what the current rules say about χ_d. The earlier Answerable Process page had instead a structural clause "met iff some alternative was resolved terminally" and no counterpart of Requirement 10; both are deliberate changes.

5.2 A matter is the obligation that survives issue replacement

An *issue* is one case or episode. A *matter* is the continuing obligation to deal with a case and whatever successor issues replace it. This distinction is what prevents procedural churn from resetting service.

Let $\preceq$ be the reflexive-transitive closure of $\to$. Matters are created in time, not retroactively. Let M_n be the matters that already exist at prefix H_n, with $M_0 = \varnothing$. After batch e_n, add

- every fresh issue $q \in \mathcal{Q}_n^+$ with $\mathsf{Par}_n(q) = \varnothing$ (a root issue), and
- every issue $q \in O_n \cup \mathcal{Q}_n^+$ for which e_n contains an accepted $\mathsf{Designate}_n(q; g)$ record.

No matter is removed from M_n. Write

$$M_\infty := \bigcup_n M_n, \qquad \beta(m) := \min\{n : m \in M_n\}$$

for the eventual set of matters and the first prefix at which matter m exists.

Lemma 2 (matter bookkeeping)**.** *$M_n \subseteq M_{n+1}$, and every $m \in M_n$ is an issue opened at some position $j < n$. Matterhood is never retroactive: $m \notin M_k$ for $k \le \beta(m) - 1$.*

Proof. Nothing is removed. A root added after e_n lies in $\mathcal{Q}_n^+$; a designated issue lies in $O_n \cup \mathcal{Q}_n^+$, hence in $\mathcal{Q}_j^+$ for some $j \le n$. The last claim is the definition of β. □

These two facts are all that the theorems of Sections 6–8 use about M_n; they are stated so that the results hold for any assignment of matters with these properties, of which the construction above is one.

Definition (live descendants of a matter)**.** For $n \ge \beta(m)$,

$$\mathsf{Live}_n(m) := \{q \in O_n : m \preceq q\}.$$

These are the currently open issues through which the matter continues.

Lemma 3 (matter continuity)**.** *If $n \ge \beta(m)$ and $\mathsf{Live}_n(m) = \varnothing$, then $\mathsf{Live}_k(m) = \varnothing$ for every $k \ge n$. More generally, every outstanding descendant of m remains part of matter m, through branching and merging.*

Proof. An existing descendant belongs to $\mathsf{Live}_n(m)$ by definition. A new descendant can only be opened as a successor of an outstanding descendant. If no such outstanding descendant exists, no later one can be created. □

6 Internal routes and when work is available

A prerequisite can block an issue while pointing to other issues that are supposed to help satisfy it. The next definitions follow those pointers through issue succession.

Definition (current internal routes)**.**

$$\mathsf{Routes}_n(d) := \{r \in O_n : \exists t \in T_d \text{ with } t \preceq r\}.$$

So if a route root $t \in T_d$ is replaced by successors, those live successors remain routes for the same prerequisite automatically.

Lemma 4 (route loss is permanent)**.** *Let d be introduced in e_j. If $n > j$ and $\mathsf{Routes}_n(d) = \varnothing$, then $\mathsf{Routes}_k(d) = \varnothing$ for all $k \geq n$.*

Proof. By Requirement 8, $T_d \subseteq O_j \cup \mathcal{Q}_j^+ \subseteq I_n$ for $n > j$, so no root in T_d opens at or after n. Any route at $k+1 \geq n+1$ is therefore either a route at k that remained outstanding, or a fresh issue in $\mathcal{Q}_k^+$ that is a successor of an outstanding descendant of some root in T_d; that parent lies in O_k and is itself a route at k. Induct on k from $\mathsf{Routes}_n(d) = \varnothing$. □

The restriction $n > j$ is needed: a root $t \in T_d$ co-opened in e_j is not outstanding at H_j, so $\mathsf{Routes}_j(d)$ may be empty while $\mathsf{Routes}_{j+1}(d) \ni t$. Since $d \in \mathcal{P}_n(q_d)$ only for $n > j$, this does not affect the waiting relation below.

For unmet prerequisites, define a directed waiting relation

$$q \rightsquigarrow_n r \quad \iff \quad \exists d \in \mathcal{P}_n(q) : \ \neg\mathsf{Met}_n(d) \ \wedge \ r \in \mathsf{Routes}_n(d).$$

Read $q \rightsquigarrow_n r$ as: "q is currently waiting through r."

Definition (issues relevant to a matter)**.** $\mathsf{Reach}_n(m)$ is the least set containing $\mathsf{Live}_n(m)$ and closed under $\rightsquigarrow_n$. In words: start with the live descendants of m and follow every chain of unmet prerequisites to the internal issues they point to. At every finite prefix this set is finite.

Definition (available work)**.**

$$\mathsf{Work}_n(m) := \{q \in \mathsf{Reach}_n(m) : \mathsf{Ready}_n(q)\} \cup \{q \in \mathsf{Reach}_n(m) : q \text{ lies on a } \rightsquigarrow_n \text{-cycle}\}.$$

A ready reachable issue is work that can proceed toward resolution. A waiting cycle also counts as work: circular internal dependence is itself something the process can act on, not a reason for permanent idleness.

Define the opportunity indicator and cumulative opportunity by

$$o_n(m) := \mathbf{1}[m \in M_n \ \wedge \ \mathsf{Work}_n(m) \neq \varnothing], \qquad \Omega_N(m) := \sum_{n<N} o_n(m).$$

Thus $\Omega_N(m)$ is simply the number of positions before N at which matter m had at least one available piece of work.

Let $a_n(m)$ be the service attention assigned to matter m at position n. We require

$$0 \leq a_n(m) \leq o_n(m), \qquad \sum_{m \in M_n} a_n(m) \leq 1, \qquad A_N(m) := \sum_{n<N} a_n(m).$$

The first inequality says attention is charged only when the matter has available work; the second is a unit total service budget. The budget is on matters, including a designated matter and the matter of its ancestor, which overlap; the earlier Answerable Process page put the budget on issues and credited matters by a maximum. The matter-grain budget is stricter under overlap and is satisfiable (Lemma 5); no theorem depends on the choice. Here attention is a service quantity, not a restriction on which structural records may be written. For example, the process may record withdrawal of a dead prerequisite while $o_n(m) = 0$; that record can restore future work without itself counting as attention to currently available work.

Structural requirement 11 (non-starvation)**.** For every $m \in M_\infty$,

$$\Omega_N(m) \to \infty \quad \Longrightarrow \quad A_N(m) \to \infty.$$

If a matter offers work infinitely often, it cannot receive only finite total attention.

Lemma 5 (non-starvation is feasible under the unit budget)**.** *Fix any injection $\iota : M_\infty \to \mathbb{N}$ that is determined at birth—for instance birth order β, with ties within one batch broken by record order—and give m the fixed share $w_m = 2^{-\iota(m)-1}$. Set*

$$a_n(m) := w_m\, o_n(m).$$

Then $0 \le a_n(m) \le o_n(m)$; $\sum_{m \in M_n} a_n(m) \le \sum_{k<K} 2^{-k-1} < 1$ for every finite M_n, however many matters appear over time and however they overlap; and $A_N(m) = w_m \Omega_N(m)$, so every matter satisfies non-starvation simultaneously. This is only a feasibility witness, not a proposed priority rule.

The final structural rule prevents a process from making a starving matter's waiting chain move forever.

Structural requirement 12 (do not rewire a starving waiting chain)**.** Suppose q is already outstanding, remains outstanding across position n, and receives at least one fresh prerequisite in e_n. Then every matter that already reaches q must currently have some available work:

$$q \in O_n \cap O_{n+1},\ \mathcal{P}_n^+(q) \neq \varnothing \quad \Longrightarrow \quad \forall m \in M_n \left(q \in \mathsf{Reach}_n(m) \Rightarrow \mathsf{Work}_n(m) \neq \varnothing\right).$$

Fresh issues may open with prerequisites. The restriction applies only to adding another prerequisite to an existing issue while some already-existing matter is stuck through that issue.

Remark (why the rule follows reachability, not ownership)*.* It is not enough to look only at matters for which q is a direct descendant. Another matter may be waiting on q through a chain of prerequisites. If that indirectly waiting matter has no available work, continually replacing q's prerequisites can keep the matter idle forever while ensuring that no single prerequisite persists. Requirement 12 blocks exactly that failure mode.

7 The main structural theorem

Lemma 6 (every outstanding issue belongs to a matter)**.** *For every $q \in O_n$, there is some $m \in M_n$ with $q \in \mathsf{Live}_n(m)$.*

Proof. Follow parent links backward from q. Parents open earlier than successors, so the chain is finite and reaches a root issue. That root became a matter when it opened. □

Lemma 7 (a reachable resolution creates an opportunity)**.** *If $q \in \mathsf{Reach}_n(m)$ and $q \in \mathsf{Res}_n$, then $o_n(m) = 1$.*

Proof. A resolving issue must be ready by Requirement 10. A ready issue in $\mathsf{Reach}_n(m)$ belongs to $\mathsf{Work}_n(m)$. □

Definition (a no-route wait)**.** For $m \in M_n$, let

$$\mathsf{NoRoute}_n(m) := \left\{ d : \begin{array}{l} d \in \mathcal{P}_n(q) \text{ for some } q \in \mathsf{Reach}_n(m), \\ \neg\mathsf{Met}_n(d), \quad \mathsf{Routes}_n(d) = \varnothing \end{array} \right\}.$$

So $d \in \mathsf{NoRoute}_n(m)$ means that somewhere along matter m's waiting structure there is an unmet prerequisite, still active, for which no live internal issue remains as a route toward satisfaction.

Theorem 2 (Persistent-Wait Theorem)**.** *Assume Requirements 4, 5, 7–10 and 12. Fix $m \in M_\infty$. If, after its birth, m remains live forever but has available work only finitely often,*

$$\exists n_0 \geq \beta(m)\ \forall n \geq n_0 :\ \mathsf{Live}_n(m) \neq \varnothing, \qquad \sup_N \Omega_N(m) < \infty,$$

then eventually one fixed no-route wait remains:

$$\exists d, N_0\ \forall n \geq N_0 :\ d \in \mathsf{NoRoute}_n(m).$$

Proof. Since $m \in M_n$ for all $n \geq \beta(m)$ and $\Omega_N(m)$ is bounded, there are only finitely many positions at which $\mathsf{Work}_n(m) \neq \varnothing$. Choose $n_1 \geq n_0$ after the last of them. Then $\mathsf{Work}_n(m) = \varnothing$ and $\mathsf{Live}_n(m) \neq \varnothing$ for every $n \geq n_1$.

1. No reachable issue resolves after n_1. If $q \in \mathsf{Reach}_n(m) \cap \mathsf{Res}_n$ with $n \geq n_1$, Lemma 7 gives $\mathsf{Work}_n(m) \neq \varnothing$, a contradiction.

2. The reachable waiting structure can only shrink: for $n \geq n_1$, $\mathsf{Live}_{n+1}(m) = \mathsf{Live}_n(m)$, $\mathsf{Reach}_{n+1}(m) \subseteq \mathsf{Reach}_n(m)$, and for every $q \in \mathsf{Reach}_{n+1}(m)$, $\mathcal{P}_{n+1}(q) \subseteq \mathcal{P}_n(q)$.

Live lineage. Let $q \in \mathsf{Live}_{n+1}(m)$. If $q \in O_n$ then $q \in \mathsf{Live}_n(m)$. Otherwise $q \in \mathcal{Q}_n^+$ by Requirement 4; $q \neq m$ because $m \in M_n$ was opened before n; so $m \preceq q$ passes through a parent $p \in \mathsf{Par}_n(q) \subseteq O_n$ with $m \preceq p$. Then $p \in \mathsf{Live}_n(m)$ resolves at n, contradicting Step 1. Conversely nothing in $\mathsf{Live}_n(m)$ resolves, so $\mathsf{Live}_n(m) \subseteq O_{n+1}$.

Prerequisites. Let $q \in \mathsf{Reach}_n(m)$. Then $q \in O_n \cap O_{n+1}$ by Step 1, and Requirement 12 with $\mathsf{Work}_n(m) = \varnothing$ gives $\mathcal{P}_n^+(q) = \varnothing$, so $\mathcal{P}_{n+1}(q) = \mathcal{P}_n(q) \setminus \mathcal{P}_n^-(q) \subseteq \mathcal{P}_n(q)$ by Requirement 7.

Routes. Let $q \in \mathsf{Reach}_n(m)$, $d \in \mathcal{P}_n(q)$ unmet at n, and $r \in \mathsf{Routes}_{n+1}(d)$. Since d was introduced before n, Requirement 8 puts $T_d \subseteq I_n$, so $r \notin T_d$ if $r \in \mathcal{Q}_n^+$. If $r \in O_n$ then $r \in \mathsf{Routes}_n(d)$, hence $q \rightsquigarrow_n r$ and $r \in \mathsf{Reach}_n(m)$ by closure. If $r \in \mathcal{Q}_n^+$, its parent $p \in \mathsf{Par}_n(r) \subseteq O_n$ satisfies $t \preceq p$ for some $t \in T_d$, so $p \in \mathsf{Routes}_n(d)$, $p \in \mathsf{Reach}_n(m)$, and p resolves at n, contradicting Step 1. Thus $\mathsf{Routes}_{n+1}(d) \subseteq \mathsf{Routes}_n(d)$, and since no route in $\mathsf{Reach}_n(m)$ resolves, in fact $\mathsf{Routes}_{n+1}(d) = \mathsf{Routes}_n(d)$.

Closure. Now let $q \in \mathsf{Reach}_{n+1}(m)$ be obtained from $\mathsf{Live}_{n+1}(m) = \mathsf{Live}_n(m) \subseteq \mathsf{Reach}_n(m)$ by an edge $q_0 \rightsquigarrow_{n+1} q$ with $q_0 \in \mathsf{Reach}_n(m)$ (induction along the closure). The edge is witnessed by some $d \in \mathcal{P}_{n+1}(q_0) \subseteq \mathcal{P}_n(q_0)$ with $\neg\mathsf{Met}_{n+1}(d)$, hence $\neg\mathsf{Met}_n(d)$ by Requirement 9, and $q \in \mathsf{Routes}_{n+1}(d) \subseteq \mathsf{Routes}_n(d)$. So $q_0 \rightsquigarrow_n q$ and $q \in \mathsf{Reach}_n(m)$.

Hence, after n_1, all reachable issues lie in the finite set $\mathsf{Reach}_{n_1}(m)$ and all their active prerequisites in the finite set $\bigcup_{q \in \mathsf{Reach}_{n_1}(m)} \mathcal{P}_{n_1}(q)$; no fresh issue and no fresh prerequisite occurrence can enter.

3. The finite structure eventually stabilizes. For $n \geq n_1$ the sets $\mathsf{Reach}_n(m)$ and $\mathcal{P}_n(q)$ (q reachable) are non-increasing, and $\mathsf{Met}_n(d)$ is non-decreasing by Requirement 9, all within one fixed finite set. Route sets of unmet reachable prerequisites are constant by Step 2. Monotone sequences in a finite set are eventually constant, so there is $n_2 \geq n_1$ after which the reachable issues, active prerequisites, satisfaction values, and route sets do not change. The waiting relation $\rightsquigarrow_n$ restricted to $\mathsf{Reach}_n(m)$ is determined by these, so it is constant too.

4. A stable finite waiting graph with no available work has a no-route wait. Fix $n \geq n_2$. The matter is still live, so $\mathsf{Reach}_n(m)$ is nonempty. No reachable issue is ready and no reachable issue lies on a $\rightsquigarrow_n$-cycle (a self-loop $q \rightsquigarrow_n q$ counts as a cycle), since either would belong to $\mathsf{Work}_n(m)$. Because $\mathsf{Reach}_n(m)$ is closed under $\rightsquigarrow_n$, the waiting graph on $\mathsf{Reach}_n(m)$ is a finite acyclic digraph and has a sink q^*. It is not ready, so some active $d \in \mathcal{P}_n(q^*)$ is unmet. If $\mathsf{Routes}_n(d) \neq \varnothing$, q^* would have an outgoing waiting edge, contrary to being a sink. Thus $\mathsf{Routes}_n(d) = \varnothing$ and $d \in \mathsf{NoRoute}_n(m)$.

5. The same wait persists. For every $n \geq n_2$ the same q^* is reachable, the same d is active on it and unmet, and $\mathsf{Routes}_n(d) = \varnothing$, all by stabilization (or, for the route set, by Lemma 4). Hence $d \in \mathsf{NoRoute}_n(m)$ for all $n \geq n_2$. □

The theorem deliberately stops here. Structural continuity can show that permanent idleness must settle on a fixed unmet wait with no internal route. It does not say why such a wait would eventually clear. For the next result we need only one liveness assumption; we do not need to classify waits as internal or external.

8 From a fixed wait to No Structural Abandonment

Additional assumption (wait responsiveness)**.** No fixed prerequisite remains a no-route wait of a matter forever: for every matter m, prerequisite occurrence d, and position N_0,

$$\exists n \geq N_0 : \ d \notin \mathsf{NoRoute}_n(m).$$

Equivalently, since $d \in \mathsf{NoRoute}_k(m)$ requires $\neg\mathsf{Met}_k(d)$,

$$\big(\forall n \geq N_0 : \ d \in \mathsf{NoRoute}_n(m)\big) \quad \Longrightarrow \quad \exists k \geq N_0 : \ \mathsf{Met}_k(d),$$

which is the form that names a mechanism. The first form is the primitive: it is exactly the negation of the Persistent-Wait conclusion, and it abstracts over why the wait clears—the prerequisite may be satisfied, withdrawn, or cease to be reachable. Coverage of outside inputs and a discipline for removing obsolete internal prerequisites are possible future sufficient conditions; the structural theory assumes only this statement and does not classify waits as internal or external.

Theorem 3 (Persistent Opportunity)**.** *Under the hypotheses of Theorem 2 and wait responsiveness, if matter m remains live forever after its birth, then*

$$\Omega_N(m) \to \infty.$$

Proof. If cumulative opportunity were bounded, Theorem 2 would produce d and N_0 with $d \in \mathsf{NoRoute}_n(m)$ for every $n \geq N_0$, which wait responsiveness denies. □

Theorem 4 (No Structural Abandonment)**.** *Add Requirement 11 (non-starvation). Then for every matter $m \in M_\infty$, at least one of the following holds (they need not be exclusive):*

$$\big(\exists N \geq \beta(m)\ \forall n \geq N : \ \mathsf{Live}_n(m) = \varnothing\big) \qquad \textit{or} \qquad A_N(m) \to \infty.$$

In words: the matter eventually closes through explicit resolutions, or it receives unbounded total attention.

Proof. After a matter is born, emptiness of $\mathsf{Live}_n(m)$ is permanent by Lemma 3. Thus either the matter eventually closes or it remains live forever. In the second case Theorem 3 gives $\Omega_N(m) \to \infty$, and non-starvation gives $A_N(m) \to \infty$. Resolution continuity makes the first case explicit rather than silent. □

If no fixed no-route wait can persist forever: resolve it, or keep servicing it.

9 One continuity principle, repeated at several levels

The individual rules are easier to remember as instances of one idea: revision may change normative content, but it may not silently break the historical relation that carries authority or obligation forward.

Part of the theory	**What it prevents**
Grounded Replay	A currently standing rule appearing without an authorization history.
New demands become issues	A demand generated by the reasoner's own rules remaining merely implicit.
Resolution continuity	An outstanding issue disappearing when the surrounding rules change.
Prospective revision	A new rule retroactively becoming the judge of an already-open case.
State continuity	A successor silently forgetting the procedural state accumulated by its parents.
Matter continuity	Renaming, splitting, or replacing issues resetting the obligation to deal with the underlying matter.
Persistent-Wait Theorem	Internal restructuring producing permanent idleness without eventually exposing one fixed unmet wait.
Non-starvation	Infinitely many opportunities for work receiving only finite total service.

Two short slogans capture the two halves:

authority must have a history obligations must have a future.

10 How this connects to criticism and substantive progress

The structural theory starts only once something is represented. It does not show that important criticism enters the record, and it does not show that attention leads to a good answer. Those are separate assumptions or theorems.

A discovery/coverage theory might supply:

> If a relevant criticism remains exposed in the world, eventually a record representing that criticism appears.

The reasoner's own current normative rules might then satisfy a responsiveness condition such as

$$\mathsf{RepCrit}_n(c) \quad \Longrightarrow \quad (\text{Registration}, c) \in \mathsf{NewDue}_{n+1}.$$

The one-step shift is just strict-prefix timing: a criticism written in e_n first becomes readable in H_{n+1}. Coverage of outside inputs can also help justify wait responsiveness; removing obsolete internal prerequisites supplies another possible source of that assumption. The present theory does not need to distinguish the two cases.

At the other end, an application-specific progress theorem might say

$$A_N(m) \to \infty \quad + \quad \text{persistent relevant evidence} \quad \Longrightarrow \quad \text{a useful accepted disposition.}$$

Nothing in the present structural theory says the critic must win. The claim is only that a protected matter cannot be erased, silently re-judged, silently reset, hidden forever behind a moving chain of prerequisites, or starved when work keeps becoming available.

The full intended composition is therefore

$$\text{persistent relevant criticism} \longrightarrow \text{represented criticism} \longrightarrow \text{newly due issue} \longrightarrow \text{matter},$$

$$\text{matter} \xrightarrow{\text{wait responsiveness}} \text{explicit closure or infinitely much opportunity},$$

$$\text{infinitely much opportunity} \longrightarrow \text{unbounded attention} \longrightarrow \text{useful disposition}.$$

Each arrow needs its own assumption or theorem.

11 Scope and proof status

What this theory does not establish. It does not establish normative correctness, good judgment, reflective openness, a final evaluator, rates of progress, or resistance to manipulating which future evidence becomes available. A self-sealing rule can have a perfectly legitimate authorization history. Structural continuity says how such a rule must be changed and how the obligations it creates persist; it does not say the rule is substantively good.

What remains fixed. The accepted initial standing state G and the eight semantic relations of Section 2 are fixed parts of the mathematical model. Their values may depend on the current represented rules, and those represented rules may themselves change. Internalizing still more of this meta-level structure is a separate problem.

Current provenance and proof status. The standing-change discipline and Grounded Replay restate the frozen Legitimate Evolution work. Resolution continuity and the basic issue/state continuity rules consolidate the subsequent Answerable Process design. The route-following construction and Theorems 2–4 are the newer part of the synthesis. The no-rewiring rule (Requirement 12) descends from the Answerable Process page's dependency discipline, which permitted a dependency record only on a frontier issue or a fresh one; that rule implies Requirement 12, which is a deliberate weakening—it also allows a prerequisite on a non-frontier issue when every matter reaching it has other work—and is the weakest gate under which the Persistent-Wait proof goes through. The ownership-only gate ("every matter for which q is a descendant has work") is refuted by the cross-matter prerequisite-rotation counterexample. The notation $\mathsf{Work}_n(m)$ is the same mathematical object previously called the actionable frontier $\mathsf{Front}_n(m)$; this is an expository rename, not a change in the definition.

Revision 2. This revision settles the modeling choices left open by the concordance of the AGENT-CONSOLIDATED checkpoint and records them where they apply: successor freshness (Requirement 5), no same-batch opening and resolution (Requirement 4), uniform strict-prefix timing (acceptance convention), resolution not gated by Permit (Section 2), grounds constrained only for changing standing records (Requirement 1), Grounded Replay over admitted occurrences (Theorem 1), Met as a primitive and Requirement 10 (Section 5), the matter-grain attention budget and the exact positive-share witness (Lemma 5), wait responsiveness in its primitive form (Section 8), the matter-bookkeeping lemma (Lemma 2), the provenance of Requirement 12 above, and the removal of an unused macro. No definition used by a theorem changed.

The theory and the theorems. Normatively continuous evolution is *defined* by Requirements 1–12 together with the compatibility assumption. The derived theorems each use a proper subset, and the document does not say otherwise anywhere:

condition	**part of the theory**	**used by**
Req. 1 (authority changes)	yes	Grounded Replay
Req. 2 (standing at opening)	yes	the bridge from Grounded Replay to issues only
Req. 3 (new demands become issues), compatibility	yes	no theorem
Req. 4, 5	yes	Lemma 3, Lemma 4, Persistent-Wait
Req. 6 (state continuity)	yes	no theorem
Req. 7–10	yes	Lemma 4 (8), Persistent-Wait
Req. 11 (non-starvation)	yes	No Structural Abandonment
Req. 12 (reach gate)	yes	Persistent-Wait
finite batches; matter bookkeeping (Lemma 2)	bookkeeping	Persistent-Wait
wait responsiveness	assumption, not part of the definition	Persistent Opportunity
Permit, Due, Continue, Designate	judgments the definition reads	no theorem reads them

Persistent Opportunity is Persistent-Wait plus wait responsiveness; No Structural Abandonment is Persistent Opportunity plus Requirement 11. Lemma 6 is used by no theorem.

Adjacent work is not superseded. This synthesis does not replace the separate Proper Exercise results about jurisdictional self-ratification and the absence of a generic no-escalation theorem; the checker/trace-agreement layer that asks whether a concrete run actually follows the represented semantics; or the evidence/uptake/answerability decomposition developed for Legitimate Improvement. Those theories answer different questions and remain separate inputs or downstream consumers.

Status. AGENT-CONSOLIDATED (29 August 2026) is the historical checkpoint from which this revision descends: the definitions and theorems were independently reconstructed and adversarially checked by an agent, with executable fixtures. NORMATIVE-CONTINUITY-MATH-SETTLED (30 August 2026): the structural mathematical specification, its principal modeling choices, theorem dependencies, satisfiability, and Lean theorem spine have been settled. This does not assert Coverage, Progress, substantive normative correctness, Proper Exercise, or realization by a concrete

reasoner. Lean-verified, under their stated abstract hypotheses: Theorems 1–4, Lemmas 3, 4, 5 and 2, the rotating countermodel in both directions, and an inhabitant of the issue-trace specification. Wait responsiveness and non-starvation are assumptions.

Next mathematical debt. The priced and clocked charge/liability machinery should appear only through a separate theorem showing that a concrete bounded reasoner satisfies these structural requirements, and stating whatever stronger finite-time or liability guarantees that construction adds. It should not be built into the structural definitions themselves.